CSE 4305
Computer Organization and Architecture

Lecture 8
Parallel Processors

Sabrina Islam
Lecturer, CSE, IUT
Contact: +8801832239897
E-mail: sabrinaislam22@iut-dhaka.edu

Intro

- **Idea**: Creating powerful computers by connecting many existing smaller ones.
- Replacing large inefficient processors with many smaller, efficient processors can deliver better performance per joule but **only if software can efficiently use them**.
- This lecture emphasizes that understanding the **underlying hardware trends** and **learning to adapt software to them** is where innovation and technical advances will occur in the years ahead.

Multiprocessor

- A computer system with at least two processors.
- In some designs, if a single processor fails in a multiprocessor with **n** processors, these systems would continue to provide service with **n – 1** processors.
- The goal of using multiprocessors is achieving **High Performance through scalability, availability and power efficiency**.

Multicore microprocessor

- A microprocessor containing multiple processors (cores) in a single integrated circuit.
- Virtually all microprocessors today in desktops and servers are multicore.
- These multicores are almost always **Shared Memory Processors (SMPs)**
    - **SMP:** A parallel processor with a single physical address space.

Task/Process-level Parallelism vs Parallel Processing Program

- **Task-level parallelism or Process-level parallelism**
  - Utilizing multiple processors by running independent programs simultaneously.
  - High throughput for independent jobs.
- **Parallel processing program:**
  - A single program that runs on multiple processors simultaneously.
  - In this lecture, we'll focus on **this** part!

Terminologies to understand before we proceed further

- Hardwares can be serial or parallel
  - **Serial Hardware**:
    - Processes one task at a time, sequentially.
    - Example: A single-core processor.
  - **Parallel Hardware**:
    - Executes multiple tasks simultaneously.
    - Uses multiple processors or multiple processing units.
    - A GPU or a multi-core CPU.

Terminologies to understand before we proceed further

- Softwares can be sequential or concurrent
  - **Sequential Software**:
    - Executes one instruction at a time in a fixed order.
    - A **C program** that reads a file, sorts its contents, and prints the results in a single thread.
  - **Concurrent Software**:
    - Multiple instructions can execute at overlapping time intervals.
    - A Java-based multi-threaded server handling multiple client connections at the same time.

		Software	
		Sequential	Concurrent
Hardware	Serial	Matrix Multiply written in MatLab running on an Intel Pentium 4	Windows Vista Operating System running on an Intel Pentium 4
	Parallel	Matrix Multiply written in MATLAB running on an Intel Core i7	Windows Vista Operating System running on an Intel Core i7

- Concurrent software can run on serial hardware or on parallel hardware.
- The same is true for sequential software.

So, we will use parallel processing program or parallel software to mean **either sequential or concurrent software running on parallel hardware.**

Parallel Processing Programs

Challenges:

1. We **must** get better performance or better energy efficiency from a parallel processing program on a multiprocessor. **Why?**
   - Because otherwise we could just use a uniprocessor.
   - Uniprocessor design techniques take advantage of instruction-level parallelism. Such innovations reduced the demand for programmers to rewrite programs for multiprocessors.
2. It is difficult to write software that uses multiple processors to complete one task faster, specially if the number of processors increases. **Why?**
   - Reporter analogy (It's in the Chapter 1 of the book referenced).
   - Scheduling, partitioning into parallel pieces, balancing the load, time to synchronize, and overhead for communication between the parties.
3. Amdahl's Law (we'll look into the details next)

Parallel Processing Programs

Amdahl's Law

$$\text{Execution time after improvement} = \frac{\text{Execution time affected by improvement}}{\text{Amount of improvement}} + \text{Execution time unaffected}$$

$$T_{new} = T_{parallelizable}/n + T_{sequential}$$

Here, n = number of processors (as it denotes the amount of improvement)

Parallel Processing Programs

Amdahl's Law

$$\text{Execution time after improvement} = \frac{\text{Execution time affected by improvement}}{\text{Amount of improvement}} + \text{Execution time unaffected}$$

$$\text{Speed-up} = \frac{\text{Execution time before}}{(\text{Execution time before} - \text{Execution time affected}) + \frac{\text{Execution time affected}}{\text{Amount of improvement}}}$$

$$\text{Speed-up} = \frac{1}{(1 - \text{Fraction time affected}) + \frac{\text{Fraction time affected}}{\text{Amount of improvement}}}$$

$$T_{new} = \frac{T_{parallelizable}}{n} + T_{sequential}$$

$$Speedup = \frac{1}{(1 - F_{parallelizable}) + \frac{F_{parallelizable}}{n}}$$

Parallel Processing Programs

Amdahl's Law

Suppose you want to achieve a speed-up of 90 times faster with 100 processors. What percentage of the original computation can be sequential?

$$90 = \frac{1}{(1-F_{parallelizable}) + \frac{F_{parallelizable}}{100}}$$

$$\Rightarrow 90 \times (1 - 0.99 \times F_{parallelizable}) = 1$$

$$\Rightarrow 90 - (90 \times 0.99 \times F_{parallelizable}) = 1$$

$$\Rightarrow 90 - 1 = 90 \times 0.99 \times F_{parallelizable}$$

$$\Rightarrow F_{parallelizable} = 89/89.1 = 0.999$$

$$therefore,$$

$$F_{sequential} = 1 - 0.999 = 0.001$$

Thus, to achieve a speed-up of 90 from 100 processors, the sequential percentage can only be 0.1%.

Parallel Processing Programs

Speed-up challenge (Scaling):

Speeding up can be challenging even if a good proportion of the program is parallelizable. But if the problem size grows, speed-up increases as well. Let's see an example.

Problem:

Suppose you want to perform two sums: one is a sum of 10 scalar variables, and one is a matrix sum of a pair of two-dimensional arrays, with dimensions 10 by 10. For now let's assume only the matrix sum is parallelizable. What speed-up do you get with 10 versus 40 processors? Next, calculate the speed-ups assuming the matrices grow to 20 by 20.

Parallel Processing Programs

(Scaling)

Problem: Suppose you want to perform two sums: one is a sum of 10 scalar variables, and one is a matrix sum of a pair of two-dimensional arrays, with dimensions 10 by 10. For now let's assume only the matrix sum is parallelizable. What speed-up do you get with 10 versus 40 processors? Next, calculate the speed-ups assuming the matrices grow to 20 by 20.

Solution:

Let's assume, performance is a function of the time for an addition, t.

Then there are 10 additions that do not benefit from parallel processors and 100 additions that do. So, the time for a single processor is 110t.

The time for 10 processor is,

$$\text{Execution time after improvement} = \frac{100t}{10} + 10t = 20t$$

The time for 40 processors is,

$$\text{Execution time after improvement} = \frac{100t}{40} + 10t = 12.5t$$

So the speed-up with 10 processors is 110t/20t = 5.5.

And the speed-up with 40 processors is 110t/12.5t = 8.8.

Thus, for this problem size, we get about 55% of the potential speed-up (actual speed-up) with 10 processors, but only 22% with 40.

Parallel Processing Programs

(Scaling)

Problem: Suppose you want to perform two sums: one is a sum of 10 scalar variables, and one is a matrix sum of a pair of two-dimensional arrays, with dimensions 10 by 10. For now let's assume only the matrix sum is parallelizable. What speed-up do you get with 10 versus 40 processors? Next, calculate the speed-ups assuming the matrices grow to 20 by 20.

Solution:

Now the second part,

When we increase the matrix, the program now takes 10t + 400t = 410t.

The execution time for 10 processors is,

$$\text{Execution time after improvement} = \frac{400t}{10} + 10t = 50t$$

The execution time for 40 processors is,

$$\text{Execution time after improvement} = \frac{400t}{40} + 10t = 20t$$

So the speed-up with 10 processors is 410t/50t = 8.2.

And the speed-up with 40 processors is 410t/20t = 20.5.

Thus, for this larger problem size, we get 82% of the potential speed-up with 10 processors and 51% with 40.

Parallel Processing Programs

From the previous example we have seen that getting good speed-up on a multiprocessor while keeping the problem size fixed is harder than getting good speed-up by increasing the size of the problem.

- **Strong scaling**
  - Speed-up achieved on a multiprocessor without increasing the size of the problem.
- **Weak scaling**
  - Speed-up achieved on a multiprocessor while increasing the size of the problem proportionally to the increase in the number of processors.
  - Memory hierarchy can interfere though, For example, if the weakly scaled dataset no longer fits in the last level cache of a multicore microprocessor, the resulting performance could be much worse than by using strong scaling.

So we have to choose the scaling approach depending on the application.

Parallel Processing Programs

Importance of Load Balancing

Problem:

To achieve the speed-up of 20.5 on the previous larger problem with 40 processors, we assumed the load was perfectly balanced. That is, each of the 40 processors had 2.5% of the work to do. Instead, show the impact on speed-up if one processor's load is higher than all the rest. Calculate at twice the load (5%) and five times the load (12.5%) for that hardest working processor. How well utilized are the rest of the processors?

Parallel Processing Programs

(Load Balancing)

Problem: To achieve the speed-up of 20.5 on the previous larger problem with 40 processors, we assumed the load was perfectly balanced. That is, each of the 40 processors had 2.5% of the work to do. Instead, show the impact on speed-up if one processor's load is higher than all the rest. Calculate at twice the load (5%) and five times the load (12.5%) for that hardest working processor. How well utilized are the rest of the processors?

Solution:

If one processor has 5% of the parallel load, then it must do 5% of 400 = 20 additions

The other 39 will share the remaining 380 additions.

Since they are operating simultaneously, we can just calculate the execution time as a maximum:

$$\text{Execution time after improvement} = \text{Max}\left(\frac{380t}{39}, \frac{20t}{1}\right) + 10t = 30t$$

The speed-up = 410t/30t = 14

Speed-up drops from 20.5 to 14.

The remaining 39 processors are utilized less than half the time, 380t/39 = 9.7t, while waiting 20t for the hardest working processor to finish.

Parallel Processing Programs

(Load Balancing)

Problem: To achieve the speed-up of 20.5 on the previous larger problem with 40 processors, we assumed the load was perfectly balanced. That is, each of the 40 processors had 2.5% of the work to do. Instead, show the impact on speed-up if one processor's load is higher than all the rest. Calculate at twice the load (5%) and five times the load (12.5%) for that hardest working processor. How well utilized are the rest of the processors?

Solution:

Now the second part,

If one processor has 12.5% of the parallel load, then it must do 12.5% of 400 = 50 additions

The other 39 will share the remaining 350 additions.

The execution time as a maximum:

$$\text{Execution time after improvement} = \text{Max}\left(\frac{350t}{39}, \frac{50t}{1}\right) + 10t = 60t$$

The speed-up = 410t/60t = 7

Speed-up drops from 20.5 to 7.

The rest of the processors are utilized less than 20% of the time (9t/50t).

That's why load balancing is important.

Hardware Multithreading

Increasing utilization of a processor by switching to another thread when one thread is stalled.

Hardware multithreading allows multiple threads to share the functional units of a single processor in an overlapping fashion to try to utilize the hardware resources efficiently. It is often confused with multiprocessing.

There are terminologies like **thread** and **process** which can be confusing as well. In the upcoming slides, we'll understand these terms and how they are different from each other.

Hardware Multithreading

Process

- A process is an independent execution unit that has its own memory space, system resources, and execution environment. It represents a running program.
- Each process has its own memory space (code, data, stack). Processes do not share memory with each other unless explicitly set up using Inter-Process Communication (IPC).
- Communication between processes is slower because they do not share memory.
- If one process crashes, it does not affect other processes.
- Example: Opening multiple instances (not tabs) of a web browser (each instance is a separate process).

Hardware Multithreading

Thread

- A thread is a lightweight execution unit within a process. Multiple threads can exist within a single process, sharing memory and resources.
- Threads within the same process share the same memory and resources, but each thread has its own stack and registers(a copy of the register file).
- Threads can communicate faster since they share the same address space.
- If a thread crashes, it can potentially crash the entire process.
- We use threads for tasks that need to run concurrently within the same application while sharing resources.
- Example: Opening multiple tabs within a single browser window (each tab runs as a thread in the same process).

Hardware Multithreading Approaches

Fine-grained multithreading

It is a technique used in processors to maximize CPU utilization by switching between multiple threads at each clock cycle. This switching is often done in a round-robin fashion, skipping any threads that are stalled at that clock cycle.

Analogy: A Restaurant with Multiple Waiters

Advantage:

- In traditional single-thread execution, if a thread experiences a stall (maybe waiting for data from memory), the CPU does nothing. Fine-grained multithreading switches to another thread at each cycle, keeping the CPU busy even when one thread is stalled.
- This improves overall efficiency because no CPU cycles are wasted.

Hardware Multithreading Approaches

Fine-grained multithreading

Disadvantage:

- Since the CPU cycles through multiple threads, even a thread that has no stalls has to wait its turn before executing its next instruction.
- This means that even if a thread could run continuously without waiting, it gets delayed by other threads sharing the CPU.
- Essentially, a single thread's execution is slower than if it had the CPU all to itself.

Hardware Multithreading Approaches

Coarse-grained multithreading

It is a technique where the CPU switches between threads only when one thread encounters a significant stall (e.g., waiting for memory access). This switching is often done in a round-robin fashion, skipping any threads that are stalled at that clock cycle.

Analogy: Checkout Counters at a Supermarket

Advantage:

- If a thread is stuck on a long stall (waiting for data from RAM), the cost of flushing and refilling the pipeline is small compared to the time wasted by waiting.
- This makes coarse-grained multithreading better suited for handling long stalls, like cache misses or memory accesses.
- Does not slow down threads unnecessarily as switching does not occur at every clock cycle.

Hardware Multithreading Approaches

Coarse-grained multithreading

Disadvantage:

- Since the CPU does not switch threads at every cycle, short stalls (waiting for a small piece of data) cause pipeline delays.
- When a stall occurs, the CPU must flush its pipeline before switching to a new thread. The new thread then has to refill the pipeline before execution resumes. This "start-up cost" makes switching inefficient for short stalls.

Hardware Multithreading Approaches

Simultaneous multithreading (SMT)

It is an advanced multithreading technique that allows a single CPU core to execute multiple threads at the same time. Unlike fine-grained and coarse-grained multithreading, where the CPU switches between threads, SMT actually runs multiple threads simultaneously in a single cycle by utilizing idle CPU resources.

Analogy: A Chef with Multiple Workstations

The key insight that motivates SMT is that multiple-issue processors often have more functional unit parallelism available than most single threads can effectively use.

SMT is always executing instructions from multiple threads, leaving it up to the hardware to associate instruction slots and renamed registers with their proper threads.

Hardware Multithreading Approaches

Simultaneous multithreading (SMT)

Advantage:

- Maximizes CPU resource usage by keeping execution units busy.
- Improves throughput by running multiple threads in parallel.
- Hides stalls (memory access delays, branch mispredictions) by allowing another thread to keep executing.

Disadvantage:

- Threads share CPU resources, thus performance gain depends on workload.
- It can cause resource contention where threads compete for execution units.
- Not always beneficial for single-threaded tasks.

Hardware Multithreading Approaches

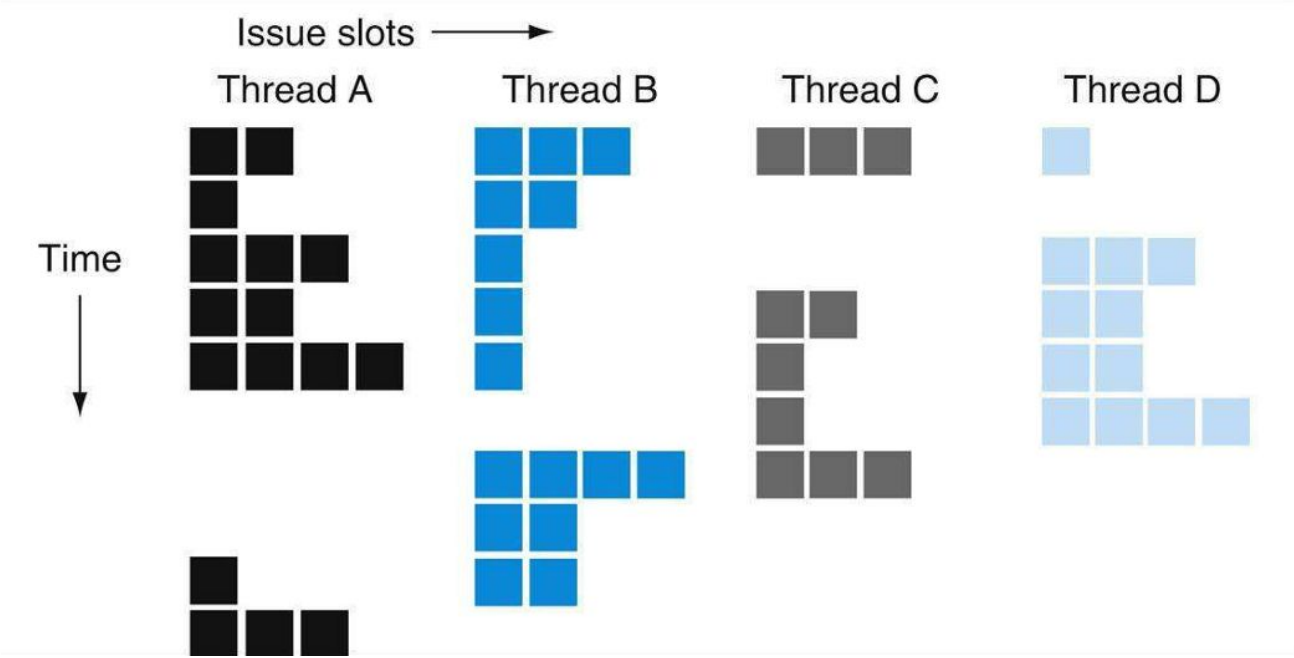
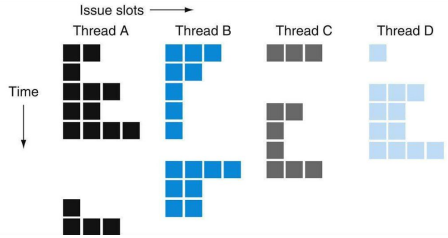


Fig: Threads running on a processor without multithreading support

Hardware Multithreading Approaches



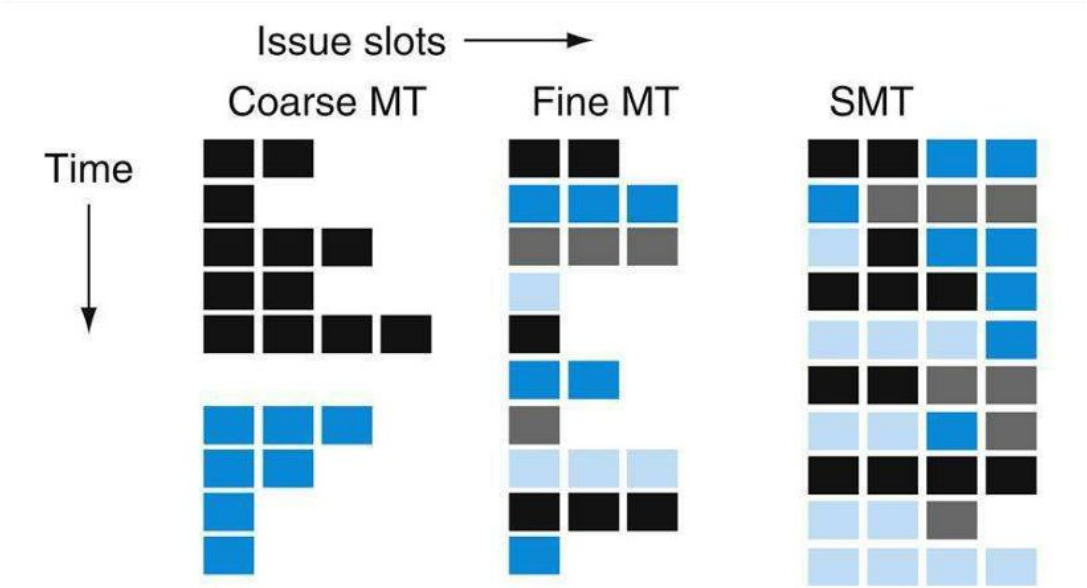


Fig: Threads running on a processor with different multithreading approaches

The average speed-up is 1.31, which is not bad given the modest extra resources for hardware multithreading. The average improvement in energy efficiency is 1.07, which is excellent.

(In energy efficiency ratio, a value of 1 means no change while values above/below indicate improvement/deterioration.)

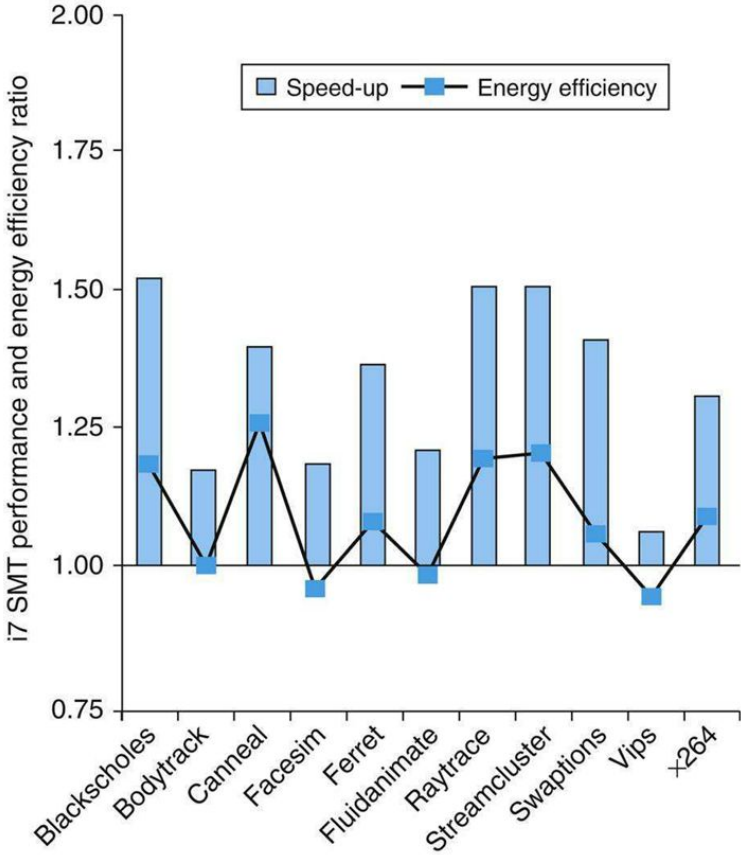


Fig: The performance and energy benefits of SMT on a single processor of the Intel Core i7 960

Reference:

1. Computer Architecture and Design RISC-V Edition, by David A Patterson and John L. Hennessy.

Thank You!